

Command-Pattern

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation

```
public class InputHandler {  
    public void handleInput() {  
        switch (keyPressed()) {  
            case BUTTON_W -> hero.jump();  
            case BUTTON_A -> hero.moveX();  
            case ...  
            default -> { ... }  
        }  
    }  
}
```

Problem: Starre Zuordnung

Auflösen der starren Zuordnung über Zwischenobjekte

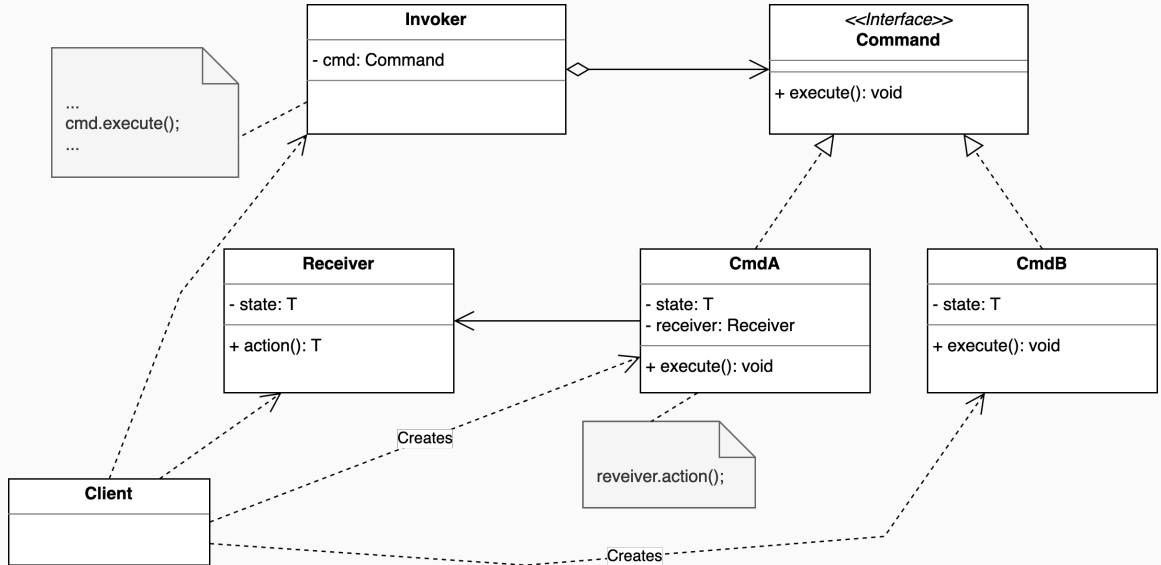
```
public interface Command { void execute(); }

public class Jump implements Command {
    private Entity e;
    public void execute() { e.jump(); }
}

public class InputHandler {
    private final Command wbutton = new Jump(hero); // Über Ctor/Methoden setzen!
    private final Command abutton = new Move(hero); // Über Ctor/Methoden setzen!

    public void handleInput() {
        switch (keyPressed()) {
            case BUTTON_W -> wbutton.execute();
            case BUTTON_A -> abutton.execute();
            case ...
            default -> { ... }
        }
    }
}
```

Command: Objektorientierte Antwort auf Callback-Funktionen



Undo

```
public class Move implements Command {  
    private Entity e;  
    private int x, y, oldX, oldY;  
  
    public void execute() { oldX = e.getX(); oldY = e.getY(); x = oldX + 42; y = oldY;  
    public void undo() { e.moveTo(oldX, oldY); }  
    public Command newCmd(Entity e) { return new Move(e); }  
}
```

```
public class InputHandler {  
    private final Command wbutton;  
    private final Command abutton;  
    private final Deque<Command> s = new ArrayDeque<>();  
  
    public void handleInput() {  
        Entity e = getSelectedEntity();  
        switch (keyPressed()) {  
            case BUTTON_W -> { s.push(wbutton.newCmd(e)); s.peek().execute(); }  
            case BUTTON_A -> { s.push(abutton.newCmd(e)); s.peek().execute(); }  
        }  
    }  
}
```

Command-Pattern: Kapselt Befehle als Objekte, um

- Aktionen entkoppelt und konfigurierbar zu machen,
 - Undo/Redo-Funktionalität zu ermöglichen,
 - Befehle zu speichern, zu loggen oder asynchron auszuführen.
-
- `Command`-Objekte haben Methode `execute()`, führen darin Aktion auf Receiver aus
 - `Receiver` sind Objekte, auf denen Aktionen ausgeführt werden (Hero, Monster, ...)
 - `Invoker` hat `Command`-Objekte und ruft darauf `execute()` auf
 - `Client` kennt alle Klassen und baut alles zusammen

Objektorientierte Antwort auf Callback-Funktionen



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

